

定时器

定时器

- 一、学习目标
 - 定时器
 - 定时器基本参数
- 二、硬件搭建
- 三、实验步骤
 - 1. 打开SYSCONFIG配置工具
 - 2. 定时器参数配置
 - 3. 写入程序
 - 5. 编译
- 四、程序分析
- 五、实验现象

一、学习目标

- 1.学习MSPM0G3507主板定时器的基本使用。
- 2.实现定时翻转LED状态。

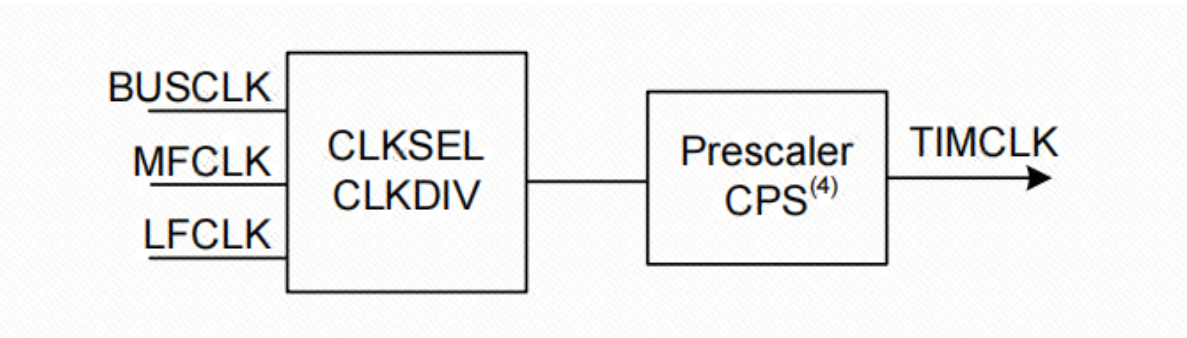
定时器

定时器是单片机中一个非常重要的外设，用于计时和产生定时中断。定时器的核心原理是通过内部计数器来记录时间，计数器每个机器周期加一，当计数器达到预设的值时，它会触发一个中断，程序可以在中断服务程序中执行某些操作，比如翻转 LED、读取传感器数据、控制电机等

定时器基本参数

1.预分频

预分频器可以将定时器的时钟(TIMCLK)频率按1到256之间的任意值分频（1到256是基于定时器分频器是8位），TIMG可以选择BUSCLK、MFCLK、LFCLK作为时钟源，可以最大8分频时钟，再经过一个8位的预分频器，最终称为定时器的计数时钟。



2.向上或向下计数模式

向上计数指的是计数器从0开始向上连续计数到自动加载值，一旦计数器计数到自动加载值，会重新从0开始向上计数并产生上溢事件。 向下计数指的是计数器从自动加载值开始向下连续计数到0，一旦计数器计数到0，会重新从自动加载值开始向下计数并产生下溢事件。

3.更新事件

更新事件是当计数器上溢或者下溢而开始新的计数周期时触发的。更新事件可以触发DMA请求，以用于在下一个计数周期开始时及时更新定时器的运行参数，特别适合用于实时控制。

二、硬件搭建

MSPM0G系列一共有7个定时器，可以分为2种类型，通用计时器（TIMG）和高级控制计时器（TIMA）。不同类型的定时器所拥有的功能数量不同，一般高级定时器的功能最多，通用定时器次之。

表 8-15. TIMx 配置

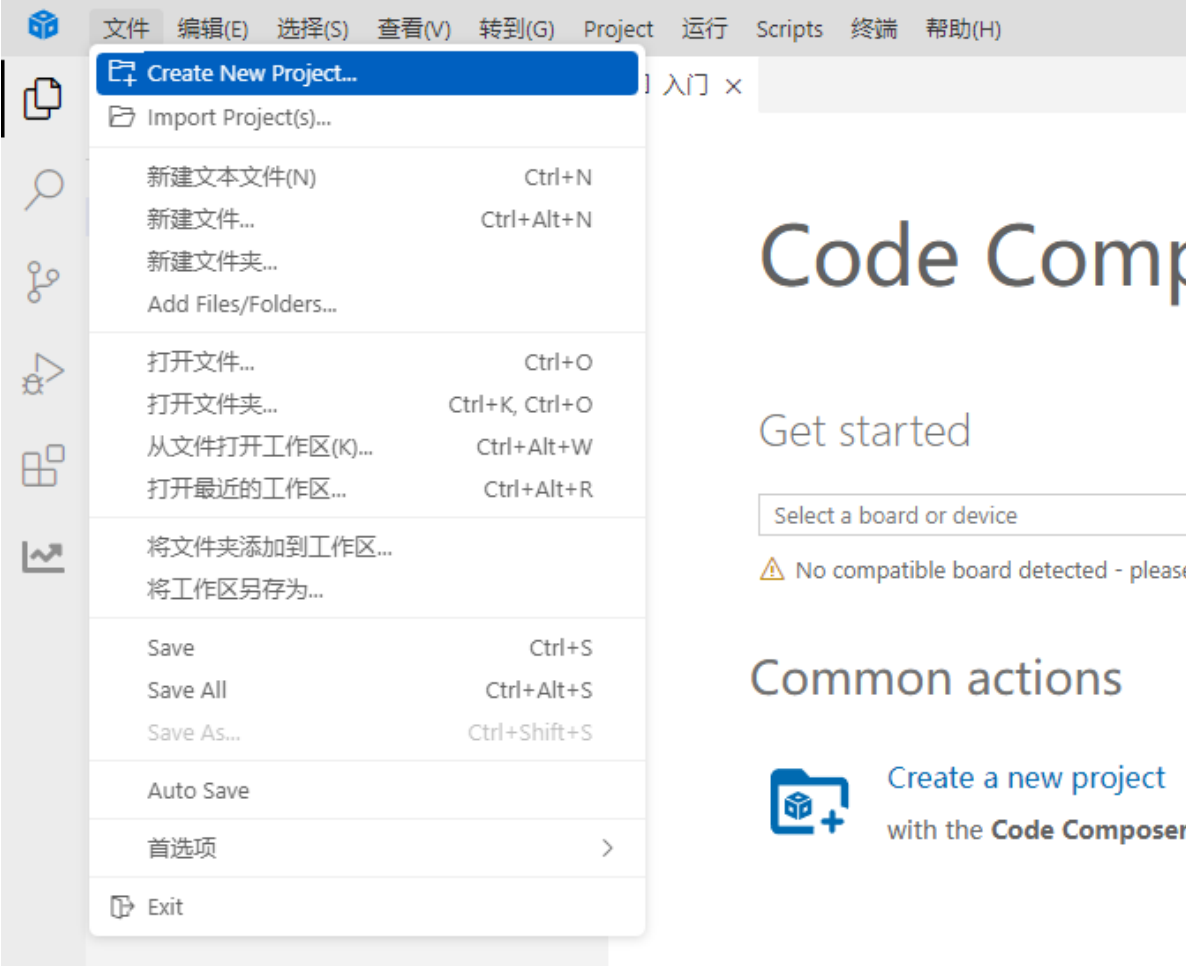
计时器名称	电源域	分辨率	预分频器	重复计数器	捕捉/比较通道	相负载	影子负载	影子CC	死区	故障	QEI
TIMG0	PD0	16 位	8 位	-	2	-	-	-	-	-	-
TIMG6	PD1	16 位	8 位	-	2	-	-	-	-	-	-
TIMG7	PD1	16 位	8 位	-	2	-	支持	支持	-	-	-
TIMG8	PD0	16 位	8 位	-	2	-	-	-	-	-	是
TIMG12	PD1	32 位	-	-	2	-	-	支持	-	-	-
TIMA0	PD1	16 位	8 位	8 位	4	是	是	是	是	支持	-
TIMA1	PD1	16 位	8 位	8 位	2	是	是	是	是	支持	-

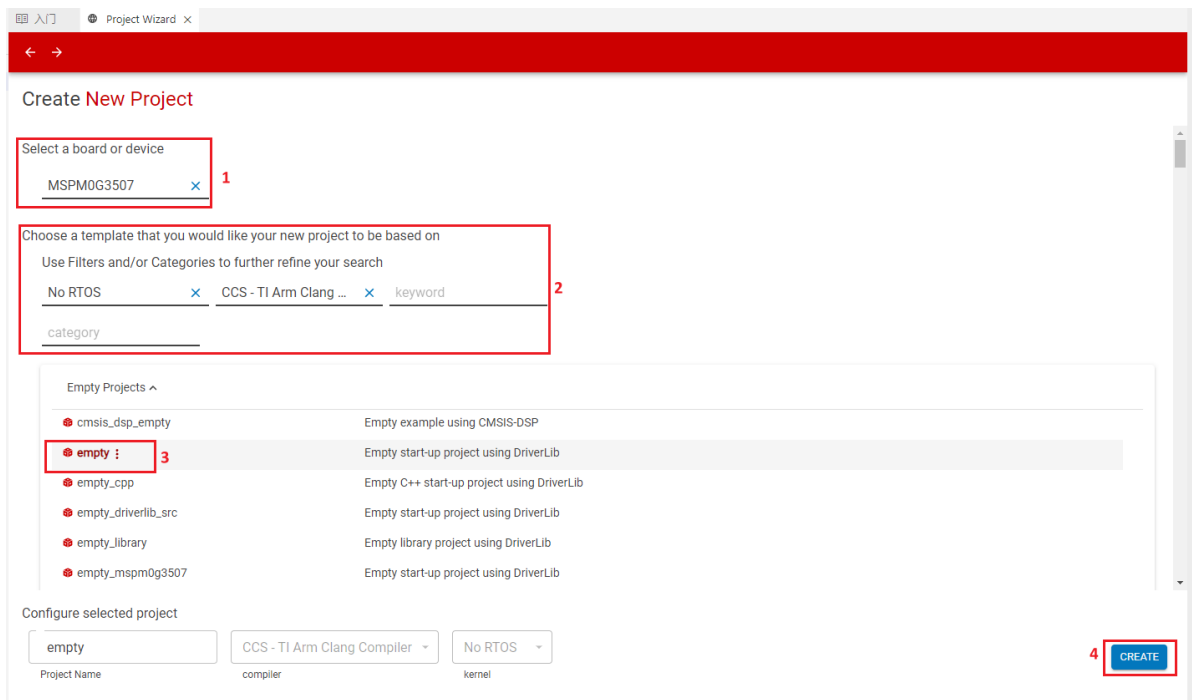
三、实验步骤

如果项目中需要定时执行或重复执行特定的任务，定时器的配置显得尤为重要。以下将介绍如何设置定时器，使其每隔1秒触发一次中断，并在中断服务函数中实现LED的状态翻转，从而达到LED亮1秒、灭1秒的效果。

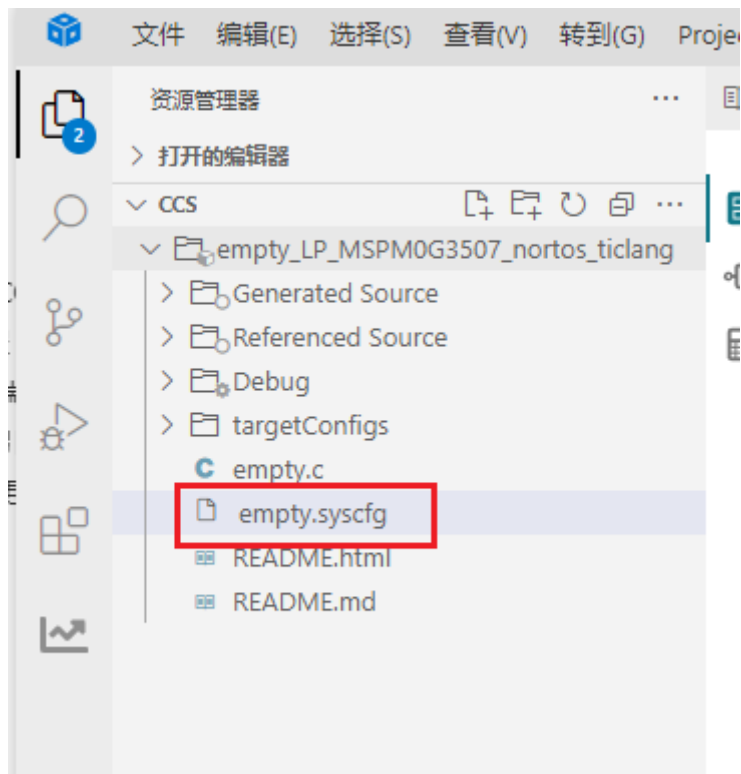
1. 打开SYSCONFIG配置工具

在CCS中新建一个空白工程 empty。





在CCS的左侧工作区中找到并打开empty.syscfg文件。



在SYSCONFIG中，左侧选择MCU外设，找到TIMER选项并点击进入。在TIMER中点击ADD，添加定时器外设。

Software > TIMER

PROJECT CONFIGURATION (1)

- Project Configur... 1/1 ✓ +

MSPM0 DRIVER LIBRARY (7)

- SYSTEM (9)**
 - Board 1/1 ✓ +
 - Configuration NVM +
 - DMA +
 - GPIO 2 ✓ +
 - MATHACL +
 - RTC +
 - SYSCCTL 1/1 ✓ +
 - SYSTICK 1/1 ✓ +
 - WWDT +
- ANALOG (6)**
 - ADC12 +
 - COMP +
 - DAC12 +
 - GPAMP +
 - OPA +
 - VREF +
- COMMUNICATIONS (6)**
 - I2C +
 - I2C - SMBUS +
 - MCAN +
 - SPI +
 - UART +
 - UART - LIN +
- TIMERS (6)**
 - TIMER** +
 - TIMER - CAPTURE +
 - TIMER - COMPARE +
 - TIMER - PWM +
 - TIMER - QEI +

TIMER (0 of 7 Added) **+ ADD** REMOVE ALL

Description

The **Timer module** allows you to configure timers in one-shot or periodic mode.

Under *Basic Configuration* users can:

- Select Timer Clock configuration
- Configure:
 - One-shot or periodic mode
 - Enable repeat mode (if supported)

Under *Advanced configuration* users can:

- Configure a timer instance to trigger other timer instances under cross-trigger configuration section.

This module supports the different types of Timers included in MSPM0 including *TIMGx*, *TIMAx*.
The following table includes a description of the features included in **MSPM0G350x**.
Check the datasheet of your selected device for more detailed information.

TIM Name	Power Domain	Resolution	Prescaler	Repeat	CC Ch	Phase Load	Shadow Load	Shadow CC	Deadb
TIMG0	PD0	16-bit	8-bit	-	2	-	-	-	-
TIMG6	PD1	16-bit	8-bit	-	2	-	-	-	-
TIMG7	PD1	16-bit	8-bit	-	2	-	Y	Y	-
TIMG8	PD0	16-bit	8-bit	-	2	-	-	-	-
TIMG12	PD1	32-bit	-	-	2	-	-	Y	-
TIMA0	PD1	16-bit	8-bit	8-bit	4	Y	Y	Y	Y

Click the **Add** button to add a **TIMER** to your design

Name TIMER_0

Selected Peripheral

Quick Profiles

2. 定时器参数配置

本案例 Timer Clock Source 定时器时钟来源选择 BUSCLK-32MHz，Timer Clock Divider 配置定时器分频，选择8分频，Timer Clock Prescaler 再配置100 预分频，得到定时器的频率为40KHz。Desired Timer Period 设置定时器周期为1S，Timer Mode 设置为周期向下计数模式，Enable Interrupts 开启0 溢出中断。LED的GPIO配置按照的LED灯课程进行。

TIMER (1 of 7 Added)

+

ADD

REMOVE ALL

✓

TIMER_0

Name

TIMER_0

Selected Peripheral

TIMG0

Quick Profiles

Timer Profiles

Custom

Basic Configuration

Clock Configuration

Timer Clock Source

BUSCLK

Timer Clock Divider

Divided by 8

Calculated Timer Clock Source

4000000

Timer Clock Prescaler

100

Calculated Timer Clock Values

Timer Clock Frequency

40.00 kHz

Timer Period Range And Resolution

25.00 μ s to 1.64 s w/ resolution of 25.00 μ s

Timer Mode

Periodic Down Counting

Desired Timer Period

1000 ms

Actual Timer Period

1.00 s

Start Timer

☒

Advanced Configuration

Interrupts Configuration

Enable Interrupts

Zero event

Interrupt Priority

Default

Event Configuration

PinMux Peripheral and Pin Configuration

Timer Peripheral

TIMG0

通过快捷键 Ctrl+S 将.syscfg文件中的配置保存。

3. 写入程序

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

int main(void)
{
    SYSCFG_DL_init();
}
```

```

//清除定时器中断标志 Clear the timer interrupt flag
NVIC_ClearPendingIRQ(TIMER_0_INST_INT_IRQN);
//使能定时器中断 Enable timer interrupt
NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);

while (1)
{

}

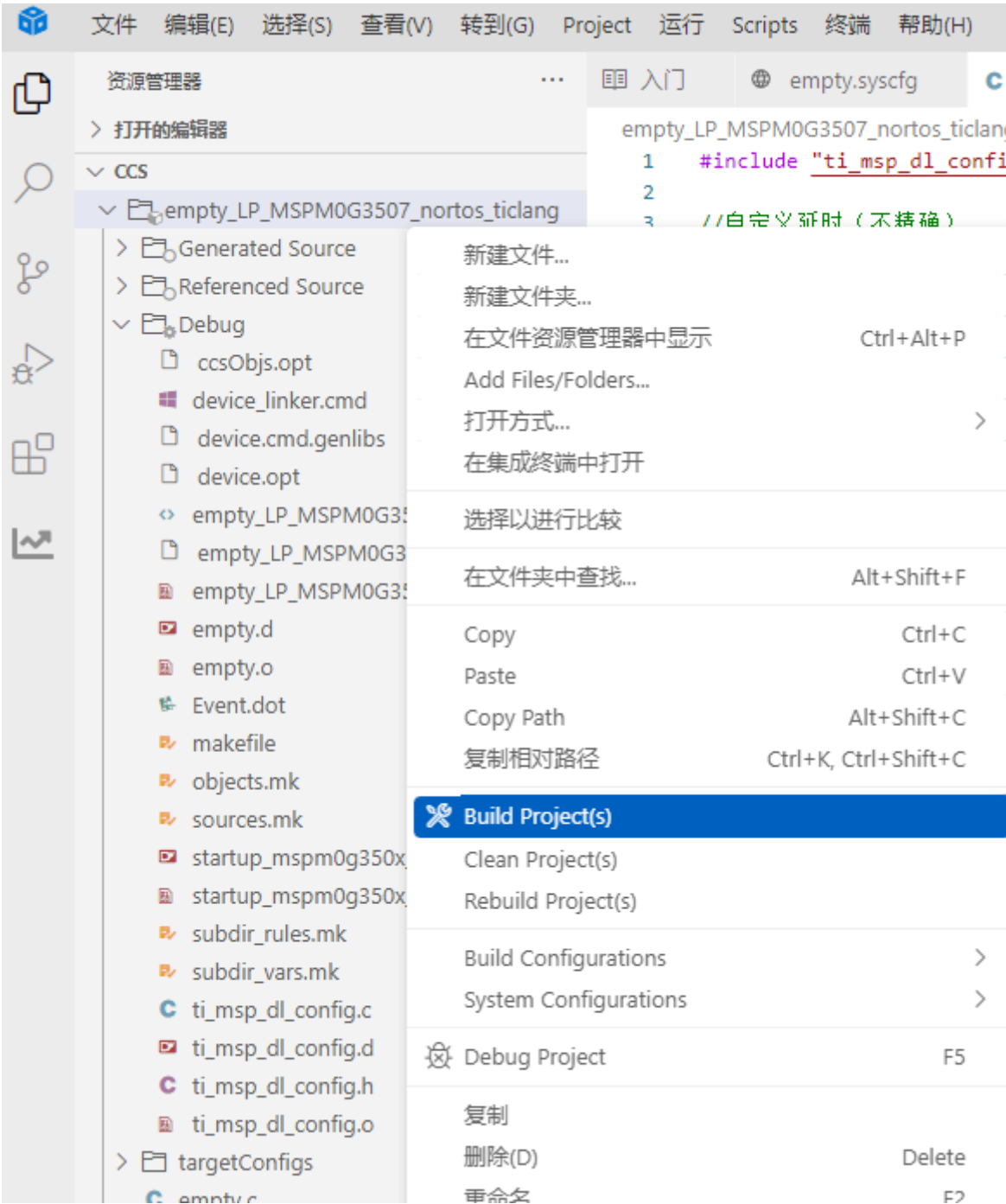
}

//定时器的中断服务函数 已配置为1秒的周期
//The timer interrupt service function has been configured to a period of 1
second
void TIMER_0_INST_IRQHandler(void)
{
    //如果产生了定时器中断 If a timer interrupt occurs
    switch( DL_TimerG_getPendingInterrupt(TIMER_0_INST) )
    {
        case DL_TIMER_IIDX_ZERO://如果是0溢出中断 If it is 0 overflow interrupt
            //将LED灯的状态翻转 Flip the LED status
            DL_GPIO_togglePins(LED1_PORT, LED1_PIN_2_PIN);
            break;

        default://其他的定时器中断 Other timer interrupts
            break;
    }
}

```

5. 编译



编译成功了，即可将程序下载到开发板。

四、程序分析

- empty.c

```

3  int main(void)
4  {
5      SYSCFG_DL_init();
6      //清除定时器中断标志 Clear the timer interrupt flag
7      NVIC_ClearPendingIRQ(TIMER_0_INST_INT_IRQN);
8      //使能定时器中断 Enable timer interrupt
9      NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);
10
11     while (1)
12     {
13
14     }
15 }
16
17 //定时器的中断服务函数 已配置为1秒的周期
18 //The timer interrupt service function has been configured to a period of 1 second
19 void TIMER_0_INST_IRQHandler(void)
20 {
21     //如果产生了定时器中断 If a timer interrupt occurs
22     switch( DL_TimerG_getPendingInterrupt(TIMER_0_INST) )
23     {
24     case DL_TIMER_IIDX_ZERO://如果是0溢出中断 If it is 0 overflow interrupt
25         //将LED灯的状态翻转 Flip the LED status
26         DL_GPIO_togglePins(LED1_PORT, LED1_PIN_2_PIN);
27         break;
28
29     default://其他的定时器中断 Other timer interrupts
30         break;
31     }
32 }

```

这是一个定时器中断服务程序，用于处理定时器溢出中断并执行相应操作。每当定时器溢出时，翻转一个 LED 的状态。

五、实验现象

程序下载完成之后，可以看到LED灯按照1秒周期的配置在闪烁。